Episode 1: Series Introduction and Lab Overview

Video Title

"Network as Code: From YAML to Running Routers in 7 Labs"

Target Length

15-20 minutes

Goal

Hook the viewer with the end result first, then walk backwards through what it took to get there. Show enough to prove this is real and valuable, but don't give away the step-by-step build process.

Pre-Recording Checklist

Before you hit record, make sure:
- ContainerLab topology is running (all 6 devices UP)
- Configs are deployed (OSPF/BGP converged)
- Ollama is running with at least 2-3 models installed
- Terminal font size is large enough for screen recording
- Two terminal windows ready: one for commands, one for output

---

SEGMENT 1: The Hook (0:00 - 2:00)

What to show: Run the fabric status script.

```
uv run python -m scripts.fabric_status --detail
```

Talking points:
- "This is a 6-node spine-leaf fabric. Two spines acting as BGP route reflectors, two leafs, two border leafs. OSPF underlay, iBGP EVPN overlay. Every one of these devices is running FRR inside ContainerLab on a single VM."
- "Every OSPF adjacency is Full. Every BGP session is Established. 16 routes on every device. And not a single line of this configuration was typed by hand."
- "This entire fabric was deployed from a YAML data model through an automated pipeline. That is what Network as Code actually looks like."

SEGMENT 2: The Problem (2:00 - 4:00)

What to show: Nothing on screen. This is you talking to the camera or over a simple slide.

Talking points:
- "80% of network outages come from misconfigurations and change management failures. That is not my number, that is from Cisco's own research on why they built their Network as Code framework."
- "The traditional workflow is: engineer SSHs into a device, types commands, hopes it works, moves to the next device, repeats. If something goes wrong, you are debugging live in production."
- "Network as Code flips that. You define what the network should look like in structured data. Automation validates it, generates the configs, deploys them, and tests the result. If something breaks, you revert a Git commit."
- "This series builds that entire stack from scratch. Seven labs, each one building on the last."

SEGMENT 3: The Series Map (4:00 - 7:00)

What to show: Open the series overview HTML in a browser.

```
open docs/nac-lab-series-overview.html
```

Walk through each lab card briefly. Do not explain how to build them. Explain what they accomplish.

Lab 1 - Data Model: "We define the entire fabric as YAML. Topology, underlay routing, overlay, VRFs, network segments. Every file has a Pydantic schema that validates it. If you put an invalid ASN or overlapping subnets, you find out before it touches a router."

-

Lab 2 - Validation: "Four layers of checks. Format, syntax, semantic, compliance. 39 rules with stable IDs. When something fails, you know exactly what layer caught it and why."

-

Lab 3 - Config Generation: "The data model feeds into a Jinja2 render engine that produces per-device FRR configurations. We also show the same thing with Ansible and Terraform so you can see when each tool fits."

-

Lab 4 - CI/CD Pipeline: "GitHub Actions. Push a feature branch, validation runs automatically. Open a PR, the pipeline generates a config diff and posts it as a comment. Merge to main, configs deploy to the devices."

-

Lab 5 - Post-Change Testing: "31 pytest tests that verify the deployment actually worked. Not just 'did the config load' but 'are the OSPF neighbors in Full state, are the BGP sessions established, can devices ping each other.'"

-

Lab 6 - Drift Detection: "Someone SSHs into a router and changes something manually. The drift engine catches it. You can auto-remediate, flag it for review, or absorb the change into the data model."

-

Lab 7 - AI Integration: "An MCP server exposes every tool we built. An AI assistant answers network operations questions using live fabric data. And it runs on a local open-source model. No cloud API required."

-

SEGMENT 4: The Money Demo (7:00 - 12:00)

This is the segment that sells the course. Show three things back to back.

Demo 1: The Validation Catch

What to show: Open a data file, break something, run validation, watch it fail.

Do NOT show the file contents in detail. Just show the edit and the result.

Exact edit: In data/services/networks.yaml, change line 14 from vrf: PRODUCTION to vrf: STAGING (a VRF that does not exist).

Talking points:
- "Let me show you what happens when someone makes a mistake in the data model."
- Make the edit on camera (just the one line)
- Run: uv run python validate.py
- "See that? SEM-04. The semantic validator caught it. The network never saw this mistake. In a traditional workflow, this becomes an outage during your next change window."
- Revert: change vrf: STAGING back to vrf: PRODUCTION

Demo 2: The Route Reflector Auto-Peering

What to show: The config diff when you change the RR flag.

Talking points:
- "This is the demo from the Cisco NaC paper. Watch what happens when I change one flag in the data model."
- Show the grep of BGP neighbors BEFORE the change
- Exact edit: In data/topology.yaml, change line 21 from route_reflector: true to route_reflector: false (spine2 loses its RR role)
- Regenerate: uv run python -m generators.python.render
- Show the grep AFTER: "Every device's BGP config just changed. Spine1 gained a client. Spine2 lost its RR role. Every leaf dropped a neighbor. One line in YAML, six device configs updated automatically."
- Revert: change line 21 back to route_reflector: true and regenerate

Demo 3: The AI Assistant

What to show: The model picker and a fabric health query.

```
uv run python -m agent.assistant --backend ollama fabric-qa "Is the fabric healthy? Give a detailed analysis."
```

Talking points:
- "This is where it gets interesting. I have multiple open-source models running on my home lab. No cloud API, no subscription, no data leaving my network."
- Show the model picker: "I have everything from 8 billion parameter models to models with hundreds of billions. Let me pick one and ask it about the fabric."

- Let the response come back
- "It just pulled live OSPF and BGP data from all six devices, sent it to a local LLM, and got back an interpreted analysis. This is not a demo environment. These are real routing protocol states from real containers."

SEGMENT 5: The Lab Environment (12:00 - 14:00)

What to show: The ContainerLab inspect output and maybe a quick vtysh session.

```
sudo containerlab inspect --topo containerlab/topology.yaml
```

Talking points:
- "The entire lab runs on a single VM. I am using Proxmox, but you could use any hypervisor or even bare metal Linux."
- "ContainerLab with FRR. No vendor licenses, no cloud costs. If you have a machine with 4GB of RAM, you can run this lab."
- "For the AI features in Lab 7, you need more RAM for the models, but Labs 1 through 6 run on minimal hardware."

Quick show of connecting to a device:

```
docker exec -it clab-nac-spine-leaf-spine1 vtysh -c "show bgp summary"
```

- "Real BGP. Real OSPF. Real routing tables. Not a simulation."

SEGMENT 6: The Ping Mesh (14:00 - 15:00)

What to show: The ping mesh script.

```
uv run python -m scripts.ping_mesh
```

Talking points:
- "Full mesh reachability. Every device can reach every other device through the overlay. 30 out of 30 pings successful. Sub-millisecond latency because it is all on the same host, but the routing path is real. Leaf1 reaches border2 through one of the two spines, selected by ECMP."

SEGMENT 7: The Close (15:00 - 17:00)

What to show: Back to you on camera or the series overview.

Talking points:
- "That is what we are building in this series. Seven labs, each one adding a layer to the automation stack."
- "The YouTube videos will walk through each lab: what it does, why it matters, and the key demos. If you want to build it yourself with the step-by-step guides, the full lab package with all seven build guides, the complete repo, and the visual diagrams is available at [your link]. It is $49."
- "Next episode, we start with Lab 1: defining the data model. I will show you how the YAML is structured,

why Pydantic validation matters, and what it looks like when the schema catches an error."
- "Subscribe so you do not miss it. I will see you in the lab."

Commands Reference (Quick Copy)

```
# Fabric status
uv run python -m scripts.fabric_status --detail

# Validation
uv run python validate.py

# Config generation
uv run python -m generators.python.render

# Deploy
uv run python -m deploy.scrapli_deploy

# Ping mesh
uv run python -m scripts.ping_mesh

# AI assistant
uv run python -m agent.assistant --backend ollama fabric-qa "Is the fabric healthy?"

# Drift detection
uv run python -m drift.detect

# ContainerLab inspect
sudo containerlab inspect --topo containerlab/topology.yaml

# Quick BGP check
docker exec clab-nac-spine-leaf-spine1 vtysh -c "show bgp summary"
```

Do NOT Show On Camera

- Full contents of YAML data files (that is paid content)
- The Pydantic schema code (paid content)
- The Jinja2 templates line by line (paid content)
- The deploy script internals (paid content)
- The validator module code (paid content)
- The step-by-step build process for any lab (paid content)

DO Show On Camera

- The outputs of running the tools
- Quick one-line edits to demonstrate concepts (but not the full file)
- The visual diagrams and series overview
- The model picker and AI responses

- ContainerLab topology and device connections
- Ping mesh and fabric status results